A Semantic Knowledge-Graph Expert System for Wine-Component Recommendation

Myron Melnyk

11 July 2026

Introduction

A wine is described by measurable **components**, such as alcohol, acidity, or sugar. In this project we do not use the exact measured numbers. Instead, for each component we only say how big it is: its **level** is *low*, *middle*, or *high*. For example, a wine can have *high alcohol*, *low sugar*, or *middle acidity*. We call one such pair – a component together with its level – a *component state* (for example `high_alcohol`). Our system takes one component state chosen by the user (for example *low chlorides*) and suggests other component states that usually appear together with it in the best wines. For each suggestion it also shows *why* it was chosen.

This is a *knowledge representation* problem, not only a statistics problem. A normal machine-learning model could guess a quality score, but it could not tell the user which things go together, or explain why. We want knowledge that is written down clearly, that a person can read, and that the program can reason about and explain. So we store the component states and the links between them as an explicit graph.

We answer three questions:

- **RQ1.** How can we turn the exact measured numbers into simple concepts such as *high alcohol*?
- **RQ2.** Which component states appear together in the best wines, and how can we store this knowledge as a graph?
- **RQ3.** Besides directly linked components, can the system also recommend a component through a *shared connection* – a middle component that both are linked to?

Background and related work

Background. To understand this report we need three simple ideas from knowledge representation:

- **Semantic network** (also called a *knowledge graph*) – knowledge stored as *nodes* joined by *links*. A node is a thing or an idea (for us, a component state such as `high_alcohol`); a link says how two nodes are related.

- **Ontology** – the same idea, but with strict rules about which kinds of nodes and links are allowed.
- **RDF triple** – one fact written as three parts, *subject – relation – object*, for example "`high_alcohol` – hasLevel – high".

We use a **semantic network** (the view taught in the course lectures [Khalid, 2026]). The other two matter because our graph can also be written as RDF triples, and could later grow into a full ontology.

Related work. Our method – building a network from how often items appear together – is the standard approach in *semantic network analysis* of *verbal fluency* data [Christensen and Kenett, 2023]. In the classic task, people name as many animals as they can; a network is then built from which animals are named together across many people, so that animals that often co-occur become connected. This is exactly the method introduced in our course [Khalid, 2026], where we built such a co-occurrence network from an animal-naming dataset. Our project applies the same idea to a different domain: instead of animals named by a person, our items are wine component states, and instead of one person's list we use one wine's set of components. The wine data comes from the UCI Wine Quality dataset [Cortez et al., 2009].

Knowledge source and representation

Source. The knowledge is derived from the UCI *Wine Quality* dataset [Cortez et al., 2009] (`https://doi.org/10.24432/C56S3T`). It contains Portuguese *Vinho Verde* samples described by 11 physicochemical features (fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol) plus an expert *quality* score. Red and white wines are provided as separate files and modelled separately.

From numbers to concepts (RQ1). Raw measurements are not knowledge; they become knowledge only once they are given meaning. For each feature we take its observed range $[\min, \max]$, split it into three equal-width intervals, and map every value to a qualitative *level* concept – *low*, *middle*, or *high*. A measurement such as "alcohol = 12.8" becomes the symbolic concept `high_alcohol`. This discretisation is the bridge from data to knowledge and is what makes the rest of the system symbolic and explainable.

Representation model (RQ2). We represent the knowledge as a **typed semantic network**. Its schema is small and explicit:

Each node in the knowledge base is now *self-describing*: it stores its type, component and level. Each edge is a *typed* `coOccursWith` relation whose `weight` records how many of the best wines contain both states. The same knowledge is expressible as RDF triples (RQ2), which we use as the conceptual reading of the graph:

Concept / class	Meaning
`Component`	a physicochemical property, e.g. *alcohol*, *chlorides*
`Level`	a qualitative value: *low*, *middle*, *high*
`ComponentState`	a component observed at a level, e.g. `high_alcohol`
`Quality`	the wine-quality class used as context (the best wines)
`Wine`	the wine type providing context: red or white

Table 1: The concepts (classes) in the model.

Relation	Domain → Range	Meaning
`hasComponent`	ComponentState → Component	which component
`hasLevel`	ComponentState → Level	at which level
`coOccursWith`	ComponentState ↔ ComponentState	jointly present in the best wines (weighted)

Table 2: The relations (links) in the model.

```
# --- as RDF triples (conceptual reading) ---
:high_alcohol   a               :ComponentState .
:high_alcohol   :hasComponent   :alcohol .
:high_alcohol   :hasLevel       :High .
:low_chlorides  :coOccursWith   :low_total_sulfur_dioxide .   # weight 18 (
    best red wines)

# --- as actually stored in knowledge_base.json ---
{ "id": "low_chlorides", "type": "ComponentState",
  "component": "chlorides", "level": "low" }
{ "source": "low_chlorides", "target": "low_total_sulfur_dioxide",
  "weight": "18", "relation": "coOccursWith" }
```

Listing 1: One node and one link, as triples and as stored JSON.

This representation is suitable because the concepts are symbolic and inspectable, the relations are typed and therefore machine-interpretable, and the graph structure directly supports the traversal-based inference described below.

System architecture

The system has three modules (Fig. 1): a Python *knowledge-acquisition* pipeline that builds the knowledge base; the *knowledge base* itself (`knowledge_base.json`); and a Next.js application containing the *inference engine* and the *user interface*. The pipeline is the single source of truth; the application only reads the knowledge base and never mutates it.

Knowledge acquisition discretises the measurements, keeps only the best wines, counts co-occurrences and emits the typed graph. **The knowledge base** stores the two typed semantic networks (red, white) plus the list of all component states. **The inference engine** (`WineRecommendationService`) answers recommendation queries by

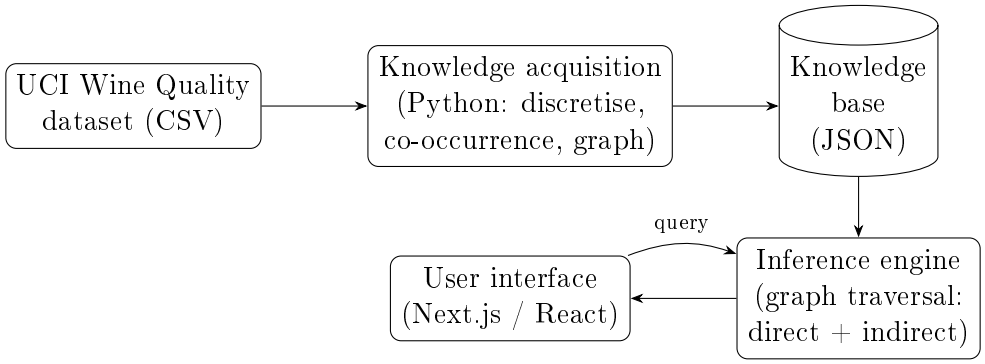


Figure 1: The three parts of the system and how the data flows between them.

traversing the graph. **The user interface** lets the user pick a component state and shows the ranked recommendations with their explanations.

Implementation

Building the knowledge base

The pipeline (`data-builder/`, Python with `pandas` and `networkx`) proceeds as follows:

1. **Discretise.** Each numeric feature is binned into *low*/*middle*/*high* by equal-width intervals over its observed range, producing a `ComponentState` label per cell.
2. **Select the best wines.** Only rows with the top quality score (`quality == 8`) are kept, so the knowledge describes what characterises *excellent* wine.
3. **Count co-occurrences.** For those wines we count how often each pair of component states appears together, giving a weighted association.
4. **Build the typed graph.** Pairs whose count exceeds a threshold become `coOccursWith` edges; isolated nodes are removed; each node is annotated with its `type`, `component` and `level`.
5. **Serialise.** The two graphs are written to `knowledge_base.json`.

The threshold is set per wine type because the number of top-quality samples differs sharply: it is 6 for red and 120 for white. As a consequence the maximum edge weight roughly equals the number of best wines available (≈ 18 for red, ≈ 175 for white); the threshold keeps only associations that are frequent *relative* to that pool.

Reasoning over the knowledge (RQ3)

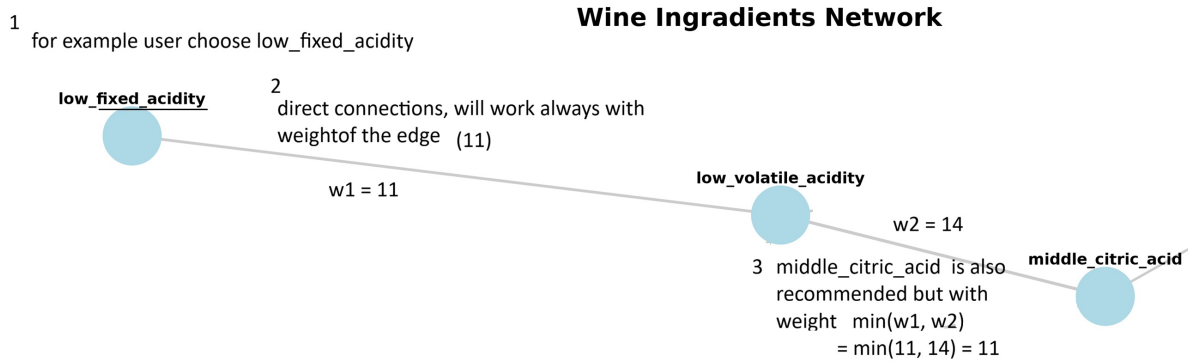
Given a query component state, the engine (`app/utils/recommendation-engine.ts`) performs graph traversal:

- **Direct inference.** Every `coOccursWith` neighbour of the query is a candidate. Its score is the edge weight normalised to the strongest edge in the graph: score $=$ round$(100 \cdot w/w_{\max})$.

- **Indirect inference (new knowledge).** If the query X co-occurs with Y and Y co-occurs with Z, the engine *infers* an association $X{-}Z$ that is **not stored in the knowledge base**, with strength $\min(w_{XY}, w_{YZ})$ and an explanation recording the intermediate node Y (see the diagram below). This is the step where the system produces knowledge beyond the stored facts.
- **Consistency filter.** The chosen component is never recommended back at another level: if the query is `low_chlorides`, the results never contain `middle_chlorides` or `high_chlorides`. This avoids telling the user to change the very thing they asked about.

Candidates are ranked by score, and the best ones are returned. In short, asking for a recommendation means asking the graph a simple question: which component states are linked to X – directly, or through one shared middle state – with the strongest links first.

How the system infers new knowledge



Here a link that was never stored (`low_fixed_acidity` to `middle_citric_acid`) is worked out from two links that were. (Weights are illustrative.)

Tools and interaction

The knowledge base is built in Python (`pandas`, `networkx`, `matplotlib`). The application is built with Next.js 16, React 19 and TypeScript; the knowledge base is a plain JSON file loaded at runtime. The user selects a component state in the browser and receives the ranked recommendations, each labelled as *direct* or *indirect*. For indirect ones, it also shows the middle state that connects them – the reason the recommendation was made.

Evaluation and discussion

Knowledge base size

Table 3 summarises the resulting knowledge base. In total there are 33 possible component states (11 components $\times$ 3 levels); the graphs retain those that survive the co-occurrence threshold.

Figure 2 shows the two graphs themselves.

Graph	Nodes	Edges	Weight range	Threshold
Red wine	14	24	7–18	6
White wine	9	13	128–175	120

Table 3: Size of the two graphs.

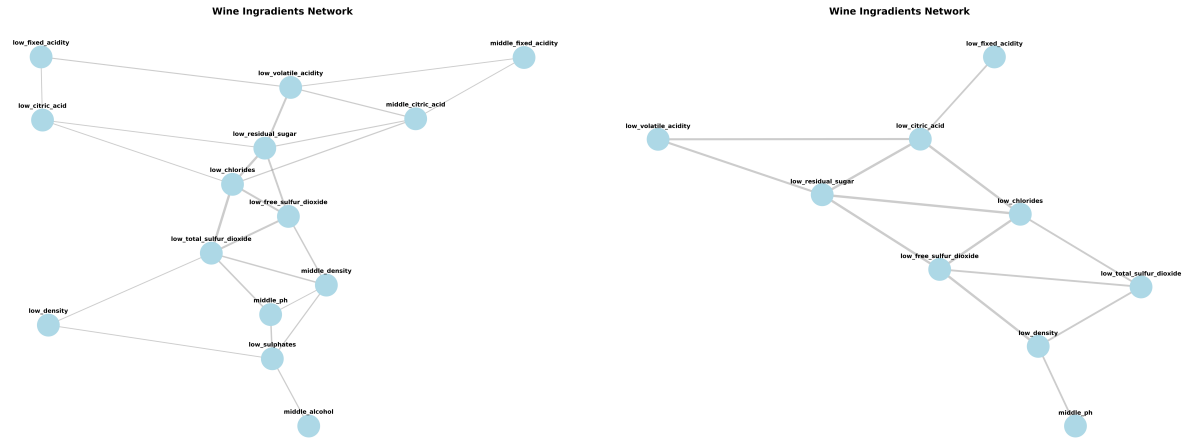


Figure 2: The two knowledge graphs – red wine (left, 14 nodes) and white wine (right, 9 nodes). Each node is a component state; each link is a `coOccursWith` relation.

Worked example: tracing the reasoning

We query the red-wine knowledge base for recommendations for `low_chlorides`. Its stored (direct) `coOccursWith` neighbours are `low_total_sulfur_dioxide` (w=18), `low_residual_suga` (17), `low_free_sulfur_dioxide` (15), `middle_citric_acid` (9) and `low_citric_acid` (7). With $w_{\max} = 18$, the engine returns the ranked list in Table 4.

Recommended state	Type	Score	Explanation
`low_total_sulfur_dioxide`	direct	100	co-occurs directly (w=18)
`low_residual_sugar`	direct	94	co-occurs directly (w=17)
`low_free_sulfur_dioxide`	direct	83	co-occurs directly (w=15)
`low_volatile_acidity`	indirect	83	inferred via `low_residual_sugar`
`middle_ph`	indirect	67	inferred via `low_total_sulfur_dioxide`

Table 4: Recommendations for `low_chlorides` (red graph). The last two are indirect – found through a shared middle state.

Figure 3 shows the same query in the web app: choosing `low_chlorides` on the red-wine graph gives exactly the list in Table 4, with both direct recommendations and inferred indirect ones.

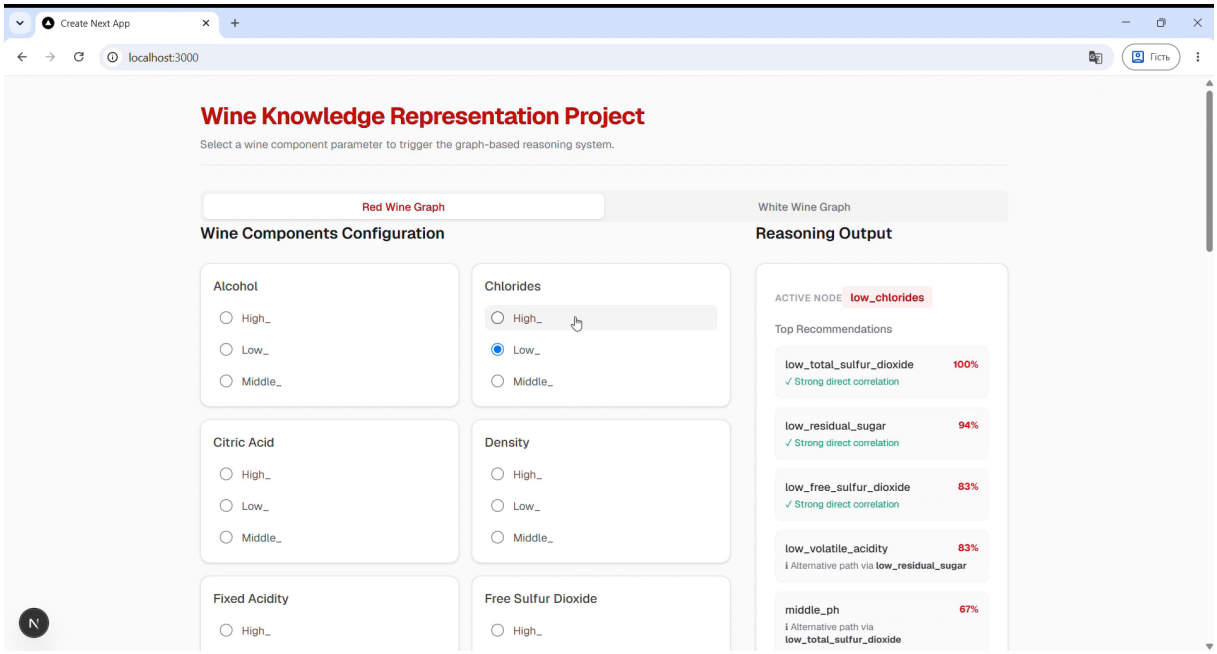


Figure 3: The same query in the web app: choosing `low_chlorides` shows the five recommendations of Table 4, each marked direct or indirect.

Discussion

Strengths. The knowledge is explicit and easy to read; every recommendation can be explained; and the same graph can be written as RDF triples.
Weaknesses. Appearing together does not mean one causes the other, so the results are not real winemaking advice. Splitting each feature into three equal parts is sensitive to outliers, and keeping only the top wines (`quality == 8`) leaves few red-wine samples, so some links rest on small counts.

Conclusion and future work

We built an expert system whose knowledge is represented explicitly as a typed semantic network of wine-component associations, and whose engine reasons over that knowledge to recommend complementary components and to infer indirect associations with explanations. This directly addresses the three research questions: measurements are turned into qualitative concepts (RQ1), those concepts and their associations are represented as a typed, self-describing graph (RQ2), and graph traversal both queries and extends that knowledge (RQ3).

Limitations and future work.

- Right now the graph only stores that two states appear together, and only for the best wines.
- It could be made richer by also comparing the best wines against the bad ones, to see what really makes a wine good.

- A useful next step is to store the knowledge as RDF/OWL and query it with SPARQL, and to add an OWL reasoner – a tool that can work out new facts on its own, instead of only walking the graph.

Declaration of generative AI use

During this work, the author used Claude Code [Anthropic, 2026], an AI coding assistant by Anthropic, as a support tool. The author checked and edited all results and takes full responsibility for the content of this report and the software.

References

Anthropic. Claude code. `https://claude.com/claude-code`, 2026. AI coding assistant, model Claude Opus 5.5. Accessed: 12 October 2026.

Alexander P. Christensen and Yoed N. Kenett. Semantic network analysis (SemNA): A tutorial on preprocessing, estimating, and analyzing semantic networks. *Psychological Methods*, 28(4):860–879, 2023. doi:10.1037/met0000463.

Paulo Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Wine Quality. UCI Machine Learning Repository, 2009. DOI: https://doi.org/10.24432/C56S3T.

Adia Khalid. Knowledge Representation, Lecture 05: Semantic Network Analysis. Lecture slides, course 392101 Knowledge Representation (S), SoSe 2026, 2026. Bielefeld University, 13 May 2026.

Note: Source code: `https://github.com/Myron5/Wine-Recommendations-System`